

Experiment No.	06
Experiment Title	Bagging, Boosting and Stacked Ensemble Models
GitHub Repository:	https://github.com/KJayasuriya/ml_laboratory/
Student Name	K. Jayasuriya
Register Number	3122247001026
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 24, 2026

1 Objective

- To understand ensemble Learning strategies: Bagging, Boosting and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability and generalization.
- To analyze the effect of ensemble methods on bias and variance.

2 Problem Statement

Perform EDA and Apply Ensemble Models for Wisconsin Diagnostic Breast Cancer Dataset. Explore hyperparameters of each ensemble model. Observe the performance of each model. Finally visualize ROC curve and Confusion matrix for each model.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset
Dataset Source	https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic
Number of Samples	569 samples
Number of Features	30 numerical Features
Number of Classes	2 - Malignant (M) and Benign (B)
Missing Values	0
Train-Test Split	Train - 455 (80%), Test - 114 (20%)

4 Exploratory Data Analysis

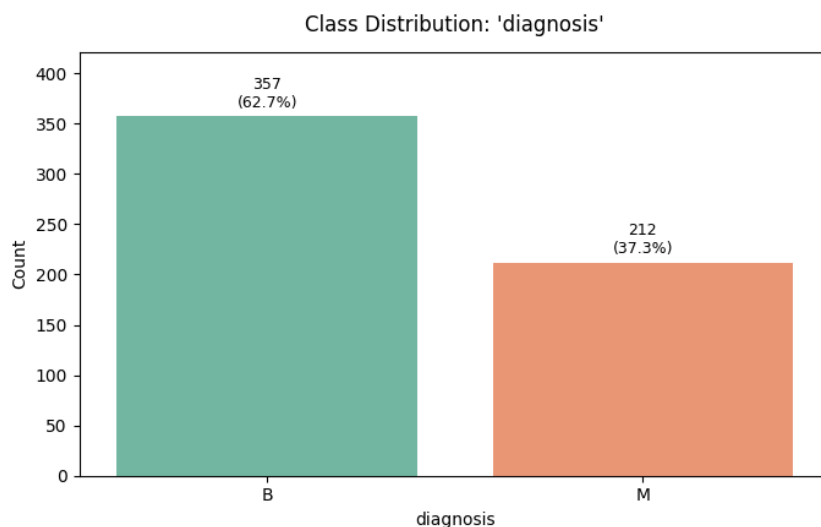


Figure 1: Class Distribution

Inference

The class distribution shows that the dataset contains 357 Benign cases and 212 Malignant cases. Therefore, the Benign class represents approximately 62.74% of the dataset, while the Malignant class represents approximately 37.26%. The classes are not perfectly balanced, but the imbalance is not extreme. Hence, both classes have sufficient samples for training and evaluation.

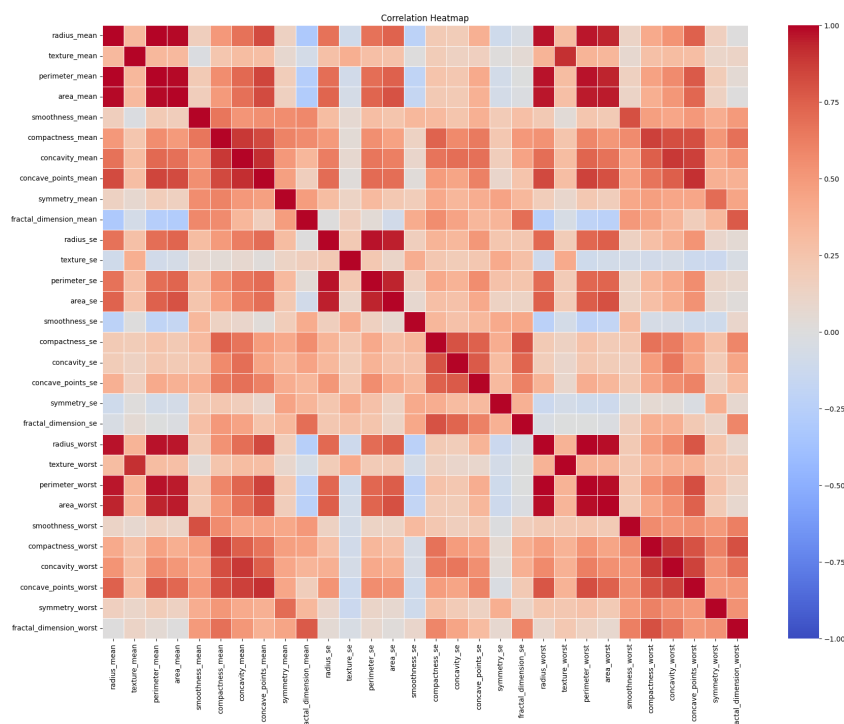


Figure 2: Correlation Matrix of Numerical Features

Inference

The correlation matrix shows the relationships among the numerical features in the breast cancer dataset. Several measurements, particularly related features such as radius, perimeter, and area, exhibit strong positive correlations. This indicates that some features contain overlapping information. Such relationships are useful for tree-based classifiers because Decision Trees and Random Forests do not require the features to be independent.

5 Data Preprocessing

- **Missing value Handling:** The dataset doesn't contain any missing values, so no imputation or removal of samples was required.
- **Duplicate Handling:** No duplicate row was found in dataset.
- **Target Variable:** The target variable is diagnosis, containing the classes Malignant (M) and Benign (B).
- **Feature Scaling:** Feature scaling was not performed. Decision Tree and Random-Forest Classifier are tree-based methods and therefore don't required normalization or standardization of numerical features.
- **Test-Train Split:** The dataset was divided into Training and testing sets using an **80:20** split with **random_state = 42**. This resulted in 455 Training and 114 testing samples.
- **Feature Engineering:** No additional feature engineering was performed because, the dataset already contains extracted numerical features.
- **Identifier column:** The implementation used in notebook removes only the **diagnosis** column before training. Therefore, the id column is also included in the feature matrix.

6 Machine Learning algorithm

Bagging

Bagging is an ensemble technique that trains multiple models on different bootstrap samples of the training data and aggregates their predictions.

Key Characteristics:

- Reduces variance.
- Suitable for high-variance models
- Independent model training.

Boosting

Boosting is an ensemble method where models are trained sequentially, and each new model focuses on correcting the errors made by previous models.

Key Characteristics:

- Reduces bias.
- Sequential Learning.
- Emphasizes difficult samples.

Stacked ensemble

Stacking combines multiple heterogeneous base models and trains a meta-learner to optimally combine their predictions

Key Characteristics:

- Uses diverse base learners.
- Meta-model learns optimal combination.
- Often achieves superior performance.

Implementation steps

1. Load and preprocess the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Split the dataset into training and testing sets (80–20).
4. Implement a Bagging classifier using a base estimator.
5. Implement Boosting classifiers (AdaBoost and/or Gradient Boosting).
6. Construct a Stacked Ensemble using multiple base models.
7. Define hyperparameter search spaces for each ensemble model.
8. Select hyperparameters using 5-fold cross-validation.
9. Evaluate all models using standard performance metrics.
10. Compare results and analyze bias–variance behavior.

7 Experimental Setup

Python Version	3.13.13
Scikit-Learn Version	1.7.5
IDE	Jupyter Notebook
Operating System	Fedora 42
Hardware	Intel Core i5 (11th gen), 8GB RAM

8 Hyperparameter Selection

Hyperparameters were selected using 5-fold cross-validation on the training dataset. The F1 score was used as the primary metric for selecting the hyperparameters, while average cross-validation accuracy was also recorded for comparison.

For the Bagging classifier, a grid search was performed over $n_{\text{estimators}} \in \{50, 100\}$, $\text{max_samples} \in \{0.8, 1.0\}$, and $\text{max_features} \in \{0.8, 1.0\}$, resulting in eight parameter combinations. The configuration with $n_{\text{estimators}} = 100$, $\text{max_samples} = 0.8$, and $\text{max_features} = 0.8$ achieved the highest average cross-validation F1 score of 0.9363 and an average cross-validation accuracy of 95.38%.

For AdaBoost, the selected configuration used 100 estimators and a learning rate of 0.5. It achieved an average cross-validation accuracy of 97.14% and an average cross-validation F1 score of 0.9602.

For Gradient Boosting, the configuration used 100 estimators, a learning rate of 0.1, and a maximum tree depth of 3. It achieved an average cross-validation accuracy of 95.16% and an average cross-validation F1 score of 0.9337.

The Stacked Ensemble consisted of SVM, Gaussian Naive Bayes, and Decision Tree as base learners, with Logistic Regression as the meta-learner. Using 5-fold cross-validation, it achieved an average cross-validation accuracy of 94.73% and an average cross-validation F1 score of 0.9257.

9 Results

Hyperparameter Evaluation Results

Table 2: Bagging Hyperparameter Evaluation

$n_{\text{estimators}}$	max_samples	max_features	Avg CV Accuracy (%)	Avg CV F1 Score
100	0.8	0.8	95.38	0.9363
50	0.8	0.8	95.38	0.9362
100	1.0	0.8	95.38	0.9359
50	1.0	0.8	95.16	0.9335
50	0.8	1.0	94.95	0.9307
100	0.8	1.0	94.73	0.9274
50	1.0	1.0	94.51	0.9239
100	1.0	1.0	94.29	0.9212

Table 3: Boosting Hyperparameter Evaluation

Model	$n_{\text{estimators}}$	learning_rate	max_depth	Avg CV Accuracy (%)	Avg CV F1 Score
AdaBoost	100	0.5	–	97.142857	0.9602
Gradient Boosting	100	0.1	3	95.164835	0.9337

Table 4: Stacked Ensemble Evaluation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM + GaussianNB + Decision Tree	Logistic Regression	94.725275	0.925711

Performance Comparison

Table 5: Performance Comparison of Ensemble Models

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
Decision Tree	97.37	100.00	93.02	96.39
Bagging	96.49	97.56	93.02	95.24
AdaBoost	96.49	97.56	93.02	95.24
Gradient Boosting	95.61	95.24	93.02	94.12
Stacking	97.37	100.00	93.02	96.39

10 Result Visualization

10.1 Confusion Matrix

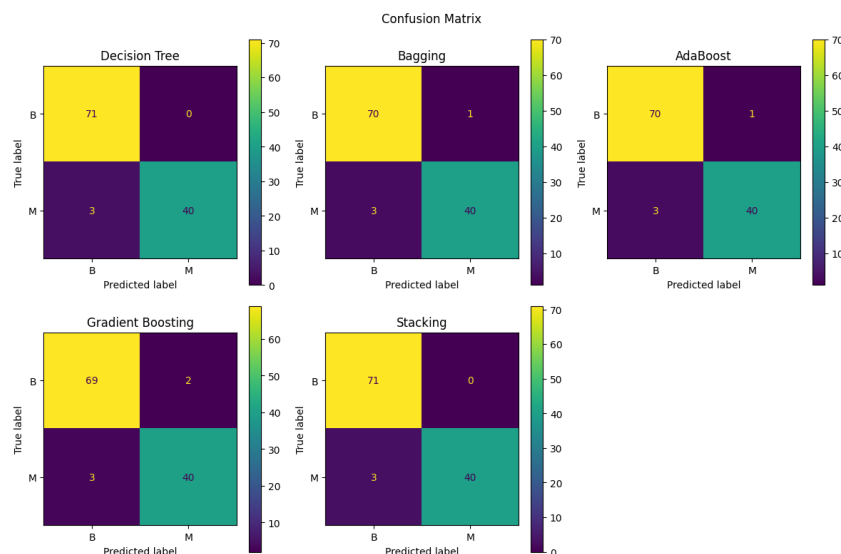


Figure 3: Confusion Matrix comparison between Base model and Ensemble Models

ROC Curve

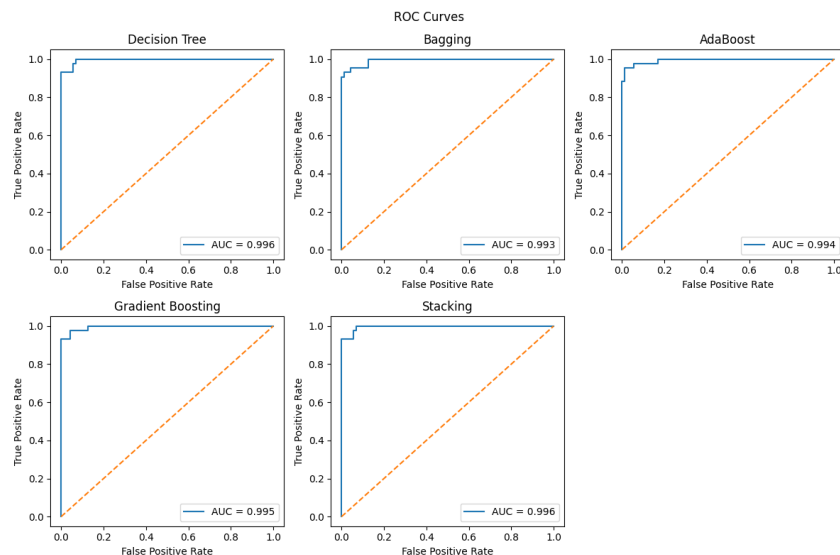


Figure 4: ROC Curves of Ensemble Models

11 Discussion

- AdaBoost achieved the highest cross-validation accuracy of 97.14% and F1 score of 96.02%.
- On the test set, Decision Tree and Stacking achieved the highest accuracy of 97.37%.

- Bagging and AdaBoost achieved 96.49% test accuracy, while Gradient Boosting achieved 95.61%.
- Stacking did not improve the test accuracy over the Decision Tree, but it achieved the same performance.
- Overall, AdaBoost performed best during cross-validation, while Decision Tree and Stacking performed best on the final test set.

12 Conclusion

Bagging, Boosting, and Stacked Ensemble models were implemented and evaluated using 5-fold cross-validation and a separate test set. AdaBoost achieved the best cross-validation performance with 97.14% accuracy and a 96.02% F1 score. Decision Tree and Stacking achieved the highest test accuracy of 97.37%. The results show that ensemble methods do not always outperform a single model, and their performance depends on the model configuration and dataset.

13 References

- [Scikit-learn: Ensemble Methods](#)
- [Bagging Classifier](#)
- [UCI Dataset](#)